

Universidad de Jaén
Departamento de Informática

Tema 6

Administración y gestión de BBDD distribuidas

Bases de Datos Distribuidas

Grado en Ingeniería Informática

CC BY NC ND

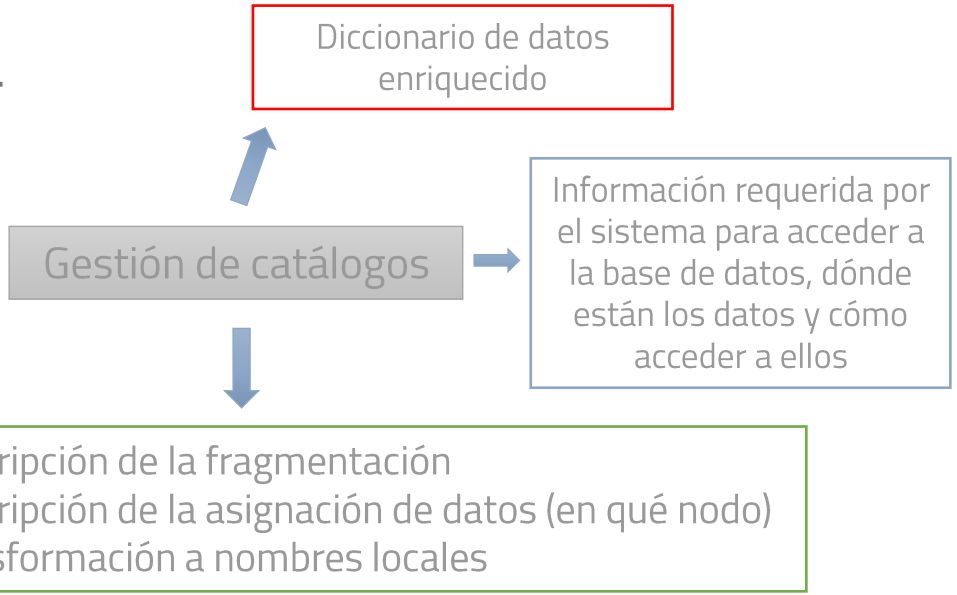
Profesor
Álvaro Labella Romero

Contenidos y bibliografía

- **Administración de BBDD distribuidas**
- Recuperación y restauración en BBDD distribuidas
- Protocolos de compromiso
- Control de concurrencia
- Seguridad en BBDD distribuidas

ADMINISTRACIÓN DE BBDD DISTRIBUIDAS

- No es solo guardar datos en varios nodos
- Abarca actividades de desarrollo, control, mantenimiento y prueba
- Es un problema técnico
  - Fragmentación, replicación, concurrencia…
- Es un problema organizativo
  - Coordinar múltiples nodos que cooperan



ADMINISTRACIÓN DE BBDD DISTRIBUIDAS

Administración según el grado de autonomía local en una BD distribuida

- Ausencia de autonomía local -> Más fácil controlar el sistema
  - Hay un nodo principal o coordinador que toma decisiones globales: dónde guardar los datos, cómo ejecutar las transacciones, cómo recuperar la información… (si se avería, tenemos un problema)
  - Administración similar al de una BD centralizada
  - Más sencillo, porque se aplica una única política de acceso a datos y reglas a seguir
- Completa autonomía local -> Más difícil coordinarlo, pero más robusto
  - Funciones del administrador global muy limitadas, la administración de la BD recae en las localidades
  - La compartición de datos es el resultado de la colaboración de las localidades no en la imposición de una autoridad central
  - Más complejo, ya que son las localidades las que deciden qué datos estarán disponibles y qué reglas deberán seguir los usuarios remotos

ADMINISTRACIÓN DE BBDD DISTRIBUIDAS

Gestión de catálogos en bases de datos distribuidas

- Los catálogos se usan para:
  1. Traslación de aplicaciones → los datos referenciados por las aplicaciones se transforman a datos físicos almacenados en los diferentes nodos (fragmentos, nodos, nombres locales…)
  2. Optimización de aplicaciones → cómo ejecutar la consulta de la forma más eficiente. Para eso, necesita saber dónde están los datos, índices, tamaño de fragmentos, coste…
  3. Ejecución de aplicaciones → verificar que los planes de acceso son válidos y que los usuarios tienen los permisos apropiados



Los catálogos se actualizan cuando se modifica la definición de los datos



Fragmentación, replicación, esquemas…

ADMINISTRACIÓN DE BBDD DISTRIBUIDAS

Contenido de los catálogos

- Descripción del esquema global:
  - Incluye nombre de las relaciones globales y de los atributos
- Descripción de la fragmentación:
  - Para fragmentación horizontal → cualificación de los fragmentos (condiciones y criterios)
  - Para fragmentación vertical → atributos que contiene cada fragmento
  - Para fragmentación mixta → árbol de fragmentación y descripción de cada nodo no hoja que representa un nivel de división de los datos
- Descripción de la asignación:
  - Dónde están ubicados físicamente los fragmentos de datos en los nodos de la red

ADMINISTRACIÓN DE BBDD DISTRIBUIDAS

Contenido de los catálogos

- Transformación a nombres locales:
  - Enlace entre datos físicos y nombres de datos locales
- Descripción de métodos de acceso:
  - Cómo se puede acceder a los datos en cada localidad: índices, hashing,…
- Estadísticas sobre la base de datos:
  - Plan de ejecución de la consulta
  - Frecuencia de acceso a cada fragmento, coste estimado de recuperación de datos…
- Información de consistencia:
  - Información sobre autorización de acceso a la base de datos por parte de los usuarios
  - Restricciones de integridad sobre los valores permitidos para los datos
  - Claves foráneas, claves primarias, restricciones de dominio…

ADMINISTRACIÓN DE BBDD DISTRIBUIDAS

Distribución de los catálogos

- Asignación y gestión de los catálogos
  - Estrictamente conectada con el grado de autonomía local
    - En sistemas centralizados puede existir sólo un catálogo
    - En sistemas descentralizados cada nodo puedes gestionar su propio catálogo o compartirlo
- Cada localidad debe ser capaz de gestionar sus datos

ADMINISTRACIÓN DE BBDD DISTRIBUIDAS

Distribución de los catálogos

- Catálogos centralizados:
  - El catálogo completo se almacena en una localidad
  - Limitaciones:
    - Las aplicaciones que no están en la localidad central deben acceder remotamente (más latencia)
    - Si la localidad central falla, todo el sistema puede verse afectado
- Catálogos completamente replicados:
  - El catálogo está replicado en cada localidad
  - Ventajas:
    - Aumenta la localidad de las aplicaciones de sólo lectura. Acceso a los metadatos de forma rápida.
  - Limitaciones:
    - La actualización debe de propagarse a todas las réplicas (inconsistencias)

ADMINISTRACIÓN DE BBDD DISTRIBUIDAS

Distribución de los catálogos

- Catálogos locales:
  - Cada nodo guarda solo la parte del catálogo que describe sus propios datos
  - Ventajas
    - Mayor eficiencia local -> cada nodo accede solo a la parte del catálogo que le interesa
- Alternativas intermedias:
  - Catálogos centralizados en una localidad y catálogos locales en el resto de localidades
  - Guardar periódicamente la información que no está localmente almacenada
    - Se mantiene la información de catálogos no locales, pero sin garantizar que siempre esté actualizada

ADMINISTRACIÓN DE BBDD DISTRIBUIDAS

Quién decide el nombre de los objetos con una **autonomía local** (tablas, atributos, claves, vistas…)

- Cada nodo puede crear sus propios objetos sin pedir permiso a un coordinador global.
  - Permite a cada usuario local crear y nombrar sus datos locales y compartirlos con otros sin necesidad de depender de un control centralizado

    Implica:

    - La definición de datos se debe realizar localmente sin necesidad de autorización o sincronización con otros nodos
    - Diferentes usuarios podrían ser capaces de asignar el mismo nombre a datos diferentes
    - Diferentes usuarios en distintas localidades deberían poder referenciar los mismos datos

ADMINISTRACIÓN DE BBDD DISTRIBUIDAS

Quién decide el nombre de los objetos con una **autonomía local** (tablas, atributos, claves, vistas…)

- Estos requisitos se logran almacenando entradas de catálogo para cada objeto:
  - Aunque un nodo pueda crear objetos con libertad, debe quedar registrado quién creó qué y dónde existe
  - Se almacena una entrada en el catálogo en la **localidad de origen** de cada objeto, hasta que el objeto se elimina completamente del sistema
    - Más fácil mantener la consistencia
  - Se almacena una entrada en el catálogo de **cada localidad en la que exista** una copia del objeto
    - Cada nodo que tiene una copia del objeto también tiene una entrada en su propio catálogo
    - Si el objeto cambia o se elimina, todas las copias del catálogo deben de actualizarse
    - Más rápido de consultar, pero más difícil de actualizar

Contenidos y bibliografía

- Administración de BBDD distribuidas
- **Recuperación y restauración en BBDD distribuidas**
- Protocolos de compromiso
- Control de concurrencia
- Seguridad en BBDD distribuidas

Recuperación y restauración en BBDD distribuidas

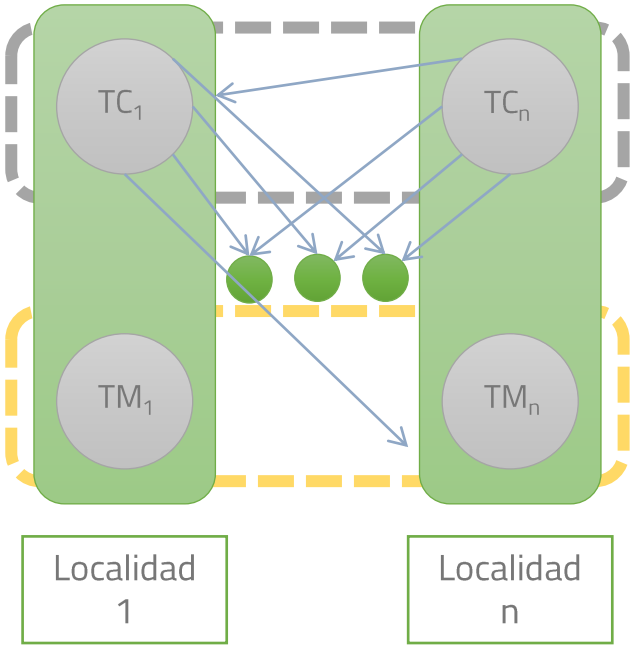
Transacción: Propiedades ACID (**Atomicidad**, Consistencia, Aislamiento, Durabilidad)

El problema -> Garantizar que una transacción siga siendo atómica cuando intervienen varios nodos

- Su ejecución atómica es más difícil de garantizar en BBDD distribuidas
  - Atomicidad: La transacción debe ejecutarse completamente o no ejecutarse en absoluto
  - Afectación a múltiples nodos. (Nodo A suma dinero, Nodo B falla y no resta)
- De garantizarla, se encarga el **gestor de transacciones**:
  - Cada localidad cuenta con su propio gestor de transacciones local
    - Se encarga de administrar las transacciones que afectan solo a los datos almacenados en ese nodo
  - Los distintos gestores de transacciones cooperan para ejecutar transacciones globales
    - Operaciones que involucran múltiples nodos. La transacción global se confirma si todos lo hacen

Recuperación y restauración en BBDD distribuidas

Estructura del sistema



Coordinador de transacciones: aparece cuando la transacción afecta a varios nodos:

- Dividir las transacciones en sub transacciones y distribuirlas en las localidades apropiadas
- Coordinar la terminación de la transacción
- Commit o abort

Gestor de transacciones: controla las transacciones locales:

- Comprueba permisos
- Solicita bloqueos
- Registra en bitácora
- Ejecuta la parte local de la transacción

Recuperación y restauración en BBDD distribuidas

Transacción: Propiedades ACID (Atomicidad, **Consistencia**, Aislamiento, Durabilidad)

El problema -> Garantizar la consistencia de los datos en un sistema distribuido cuando se producen fallos en los nodos

- Tipos de fallos en una configuración distribuida:
  - El fallo total de una localidad: un nodo entero deja de funcionar
  - La interrupción de una línea de comunicación: la conexión entre nodos se pierde
  - Pérdida de mensajes: mensajes de confirmación, actualización o sincronización
  - Fragmentación de la red: localidades divididas en grupos que no pueden comunicarse entre sí
- La consistencia supone:
  - Detectar el fallo -> protocolos de detección de fallos (Ej. envío de señales periódicas de los nodos, Heartbeat Protocol)
  - Reconfigurar el sistema -> redirigir el tráfico, aislar nodos…
  - Recuperar el sistema -> rehacer transacciones, reintentar la comunicación entre nodos…

Recuperación y restauración en BBDD distribuidas

- Procedimiento de reconfiguración:

  Se ha detectado un fallo en la localidad L_1:

  - Si en L_1 se almacena información replicada, actualizar el catálogo para no hacer referencia a la copia de L_1
  - Si en el momento del fallo existían en L_1 transacciones activas, deben abortarse
  - Si L_1 es el distribuidor central de algún subsistema (tiene un papel en la administración y coordinación de los nodos), elegir un nuevo distribuidor

Recuperación y restauración en BBDD distribuidas

- Procedimiento de recuperación:

  Cuando la localidad L_1 se recupera:

  - Iniciar procedimiento de actualización de sus tablas del sistema para reflejar los cambios
    - Se revisa el catálogo para reflejar los cambios que se produjeron mientras L_1 estaba inactivo
  - Si L_1 tenía réplica de datos, obtener los valores actuales y asegurar que recibe actualizaciones futuras

  Mientras el sistema se recupera, pueden estar actualizándose los datos que se procesan por lo que se deben aplicar mecanismos de concurrencia para evitar inconsistencias

    - Ej. Se actualiza la tabla Clientes en otro nodo mientras L_1 se recupera

Recuperación y restauración en BBDD distribuidas

- Procedimiento de recuperación:

  Posibles soluciones a la actualización mientras el sistema se recupera:

  - Suspender temporalmente todas las operaciones del sistema mientras L_1 se recupera: **inaceptable** para las aplicaciones
  - Representar las tareas de recuperación como una serie de transacciones: el subsistema de control de concurrencia y el manejo de transacciones reintegran L_1

  Cuando se recupera una línea de comunicación:

  - Enviar un mensaje a todas las localidades de la red informando que la línea se recuperó

Contenidos y bibliografía

- Administración de BBDD distribuidas
- Recuperación y restauración en BBDD distribuidas
- **Protocolos de compromiso**
- Control de concurrencia
- Seguridad en BBDD distribuidas

Protocolos de compromiso

Garantizar la atomicidad

- Todas las localidades en las que se ejecute una transacción T, tienen que coincidir en el resultado final de dicha ejecución



- El **coordinador de transacciones** debe ejecutar un protocolo de compromiso para tomar una decisión única para todos los participantes: si todos los nodos pueden confirmar la transacción, se hace un **commit** global, si algún nodo fallo o rechaza la transacción, **abort** global.
- Ejemplo: Una transacción realiza transferencias bancarias entre dos bancos distintos.
  - Si uno de los bancos registra el débito, pero el otro no registra el crédito, la base de datos queda inconsistente.
  - Un protocolo de compromiso asegura que ambos bancos acuerden si la transacción se confirma o se aborta

Protocolos de compromiso

Compromiso de 2 fases

- Sea T una transacción que se inició en la localidad L_i
- Sea C_i el coordinador de transacciones de L_i
- Fase1: tomar decisión; Fase 2: implementar la decisión
  - Después, cuando T termina de ejecutarse: realizar *commit* o abortar

Fase 1

- C_i añade el registro <*preparar T*> a la bitácora y lo graba en memoria estable (disco duro, RAID…)
- C_i envía un mensaje de *preparar T* a todas las localidades en las que se ejecutará T
- Cada gestor de transacciones determina si está dispuesto a ejecutar la parte de T que le corresponde

Protocolos de compromiso

Compromiso de 2 fases

- Si no está dispuesto:
  - Añade el registro <*no T*> a la bitácora y envía a C_i un mensaje de *abortar T*
- Si está dispuesto:
  - Añade el registro <*T lista*> a la bitácora
  - Graba en memoria estable los registros de bitácora que corresponden a T
  - Envían a C_i un mensaje de *T lista*

Fase 2

- Cuando todas las localidades han respondido al mensaje *preparar T* enviado por C_i

Protocolos de compromiso

Compromiso de 2 fases

- Si C_i recibió el mensaje *T lista* de todas las localidades que participan, T puede ejecutarse:
  - Añade el registro <*ejecutar T*> a la bitácora y se graba en memoria estable
  - C_i envía un mensaje *ejecutar T* a las localidades participantes que lo registran en la bitácora
- En caso contrario, la transacción debe abortarse:
  - Añade el registro <*abortar T*> a la bitácora y se graba en memoria estable
  - C_i envía un mensaje *abortar T* a las localidades participantes que lo registran en la bitácora

Final

- Las localidades envían a C_i un mensaje de *reconocimiento T*
- Cuando C_i recibe ese mensaje de todas las localidades, añade el registro <*T completo*> a la bitácora

Protocolos de compromiso

Consideraciones

- Una localidad puede abortar T de manera incondicional antes de enviar *T lista* a C_i
- *T lista* es una promesa de la localidad de cumplir con la orden de C_i de ejecutar o abortar T
- La promesa sólo puede hacerse si la información requerida está en almacenamiento estable
- Se requiere unanimidad para ejecutar una transacción
- C_i puede abortar T de forma unilateral
- El destino de T se fija cuando C_i lo registra en la bitácora y lo graba en memoria estable
  - A partir de este momento, la decisión es irreversible.

Protocolos de compromiso

Manejo de fallos

- Fallo de una localidad participante

  Cuando una localidad L_k se recupera de un fallo, debe examinar la bitácora:

  - Si la bitácora contiene un registro <*ejecutar T*>, L_k ejecuta *rehacer(T)*
    - Indica que, antes del fallo, en el nodo había la orden de confirmar la transacción
    - El nodo debe de rehacer la transacción al recuperarse
  - Si la bitácora contiene un registro <*abortar T*>, L_k ejecuta *deshacer(T)*
    - Indica que, antes del fallo, el nodo había recibido una orden para deshacer la transacción
    - El nodo deshace los cambios que la transacción pudo haber hecho antes del fallo

Protocolos de compromiso

Manejo de fallos

- Fallo de una localidad participante
  - Si la bitácora contiene un registro <*T lista*>, L_k consulta con C_i:
    - Si C_i está activo, notificará a L_k la ejecución o aborto de T
    - Si C_i está inactivo, L_k envía un mensaje de *consulta de estado de T*, a todas las demás localidades
      - Al recibir el mensaje, las localidades consultarán su bitácora y notificarán a L_k el resultado
      - Si ninguna localidad dispone de la información, la decisión sobre T se pospondrá
  - Si la bitácora de la localidad no contiene registros de control, L_k debe ejecutar *deshacer(T)*
    - La localidad revisa su bitácora después del fallo
    - Si no hay ningún registro sobre T significa que la localidad nunca recibió instrucciones sobre si confirmar o abortar -> revertir cualquier cambio parcial que se haya hecho antes del fallo

Protocolos de compromiso

Manejo de fallos

- Fallo del coordinador

  Si C_i falla, las localidades participantes deciden el destino de T

  - Si una localidad contiene un registro <*ejecutar T*> en su bitácora, T debe ser ejecutada
  - Si una localidad contiene un registro <*abortar T*> en su bitácora, T debe ser abortada
  - Si una localidad no contiene un registro <*T lista*> en su bitácora, C_i no pudo decidir ejecutar T
  - Si ninguno de los casos anteriores, hasta que C_i se recupere es imposible determinar si hubo una decisión y cuál fue

Protocolos de compromiso

Manejo de fallos

- Fallo del coordinador
  - Consecuencias:
    - T puede continuar reteniendo recursos del sistema, esperando una decisión final
    - Otras transacciones que dependen de T pueden haber sido forzadas a esperar a T
    - Los nuevos datos no están disponibles ni en C_i, ni en otras localidades activas
    - Cuanto mayor sea el tiempo que C_i esté caído, más datos y transacciones no disponibles



Problema de bloqueo

Protocolos de compromiso

Manejo de fallos

- Fallo de una línea de comunicación

  Todos los mensajes enrutados, no llegan intactos a su destino:

  - Desde el punto de vista de las localidades conectadas, parece que el fallo es de las otras localidades
  - Se puede aplicar el esquema anterior (fallo de una localidad participante)

- Fragmentación de la red → comunicación entre localidades interrumpida generando grupos aislados:
  - Si el coordinador y sus participantes quedan en un grupo:
    - El fallo no tiene efecto sobre el protocolo de compromiso porque todos los nodos involucrados se pueden comunicar entre sí y éstos con el coordinador
  - Si el coordinador y sus participantes quedan distribuidos en varios grupos:
    - Se perderán los mensajes entre el coordinador y los participantes
    - Situación equivalente a la interrupción de líneas de comunicación

Contenidos y bibliografía

- Administración de BBDD distribuidas
- Recuperación y restauración en BBDD distribuidas
- Protocolos de compromiso
- **Control de concurrencia**
- Seguridad en BBDD distribuidas

CONTROL DE CONCURRENCIA

Ejemplo: aerolínea con un sistema distribuido de reserva con servidores en diferentes países y datos de vuelos replicados en diferentes servidores

- Un vuelo a Madrid – Atenas tiene 1 asiento disponible
- Dos pasajeros intentan reservar el asiento al mismo tiempo
- Ambos pasajeros reservan al mismo tiempo y los servidores procesan la solicitud
- Cada servidor cree que puede reservar el asiento y lo confirman a ambos pasajeros
- **Se han vendido dos billetes para 1 solo asiento**
- **SOLUCIÓN -> PROTOCOLOS DE BLOQUEO**

Control de concurrencia

Protocolos de bloqueo

- Controlan el acceso concurrente a los datos y garantizan la consistencia
- Bloqueo compartido: se utiliza con operaciones de lectura
  - Varias transacciones pueden acceder al dato en modo lectura
- Bloqueo exclusivo: se utiliza con operaciones de actualización
  - Solo una transacción puede tener un bloque exclusivo sobre un dato
  - Ninguna otra transacción puede leer o modificar un dato hasta que el bloqueo se libere

Control de concurrencia

- **Protocolos de bloqueo: Esquema sin repetición**
  - Los datos no se repiten en el sistema
  - Cada localidad mantiene un gestor de bloqueos que administra las solicitudes de bloqueo y desbloqueo de los datos almacenados en esa localidad
  - Cuando una transacción desea bloquear el dato Q en la localidad L_i:
    - Envía un mensaje a L_i solicitando el bloqueo
    - Si Q tiene un bloqueo incompatible, la solicitud se posterga
    - Si se puede atender la solicitud, el gestor de bloqueo concede el bloqueo a la localidad que lo originó
  - Ventajas:

    Fácil de implementar: requiere un mensaje para bloqueo y otro para desbloqueo
  - Inconvenientes:

    Manejo de bloqueos más complejos: cuando las solicitudes no se hacen en una única localidad

Control de concurrencia

- **Protocolos de bloqueo: Esquema de coordinador único**
  - El sistema mantiene un único gestor de bloqueos ubicado en una de las localidades
  - Una transacción de cualquier nodo tiene que enviar la solicitud al coordinador único
  - Si el bloqueo puede realizarse → El gestor de bloqueos envía el mensaje correspondiente a la localidad donde se originó la solicitud
  - En caso contrario → la solicitud se posterga hasta que pueda atenderse
  - La transacción puede leer un dato en cualquiera de las localidades que mantengan una copia
  - La escritura de un dato tiene que efectuarse en todas las localidades que mantengan una copia
  - Ventajas:
    - Sencillez de implementación
    - Sencillez del manejo de bloqueos
  - Inconvenientes:
    - Cuello de botella
    - Vulnerabilidad

Control de concurrencia

- **Protocolos de bloqueo: Esquema de coordinadores múltiples**
  - Manejo de bloqueos distribuidos entre varias localidades
  - Cada gestor administra solicitudes para un subconjunto de datos reduciendo la carga de un solo nodo
  - Reduce la posibilidad de cuello de botella, pero complica el manejo de bloqueos
    - Si una transacción necesita bloquear datos gestionados por distintos coordinadores, debe de comunicarse con cada uno de ellos.
    - El protocolo de comunicación debe garantizar que no haya conflicto entre los diferentes coordinadores

Control de concurrencia

Protocolos de bloqueo: Protocolo de mayoría

- El sistema mantiene un gestor de bloqueos en cada localidad
- Cada gestor administra los bloqueos de los datos y copias almacenadas en esa localidad
- Cuando una transacción requiere bloquear un dato Q:
  - Debe enviar la solicitud a más de la mitad de las n localidades en las que existe una copia de Q
  - Cada localidad determina si el bloqueo puede realizarse inmediatamente
  - En caso contrario, la respuesta se posterga hasta que pueda atenderse
  - La transacción no opera sobre Q hasta que se hayan bloqueado más de la mitad de las copias

Contenidos y bibliografía

- Administración de BBDD distribuidas
- Recuperación y restauración en BBDD distribuidas
- Protocolos de compromiso
- Control de concurrencia
- **Seguridad en BBDD distribuidas**

SEGURIDAD EN BBDD DISTRIBUIDAS

Protección localidad a localidad

- Cuando dos localidades se comunican, hay que asegurar:
  - Al otro lado de la línea está la localidad deseada:
    - Establecer un protocolo de identificación entre localidades remotas
    - Las localidades se "presentan" las unas a las otras, intercambiando palabras claves
  - Que no se lean o manipulen los mensajes intercambiados entre las localidades:
    - Los mensajes son enrutados por caminos que involucran nodos intermedios
    - Exigir que todas las etapas intermedias se lleven a cabo después de una identificación y bajo mecanismos seguros, parece ineficiente e inapropiado

SEGURIDAD EN BBDD DISTRIBUIDAS

Protección localidad a localidad

Solución: Usar criptografía

- Los mensajes son codificados en mensajes cifrados en la localidad emisora
- Se transmiten a través de la red
- Se decodifican en la localidad receptora
- La codificación incluye transformación de grupos de bits, transposiciones, “confusiones” y operaciones lógicas reversibles
- Las reglas para codificar y decodificar se denominan claves del sistema criptográfico
- Se requiere que el emisor y el receptor de la transmisión estén de acuerdo con la clave

SEGURIDAD EN BBDD DISTRIBUIDAS

Identificación del usuario

- Cuando un usuario se conecta al sistema, debe ser identificado
- Aspecto crítico para preservar la seguridad
- En un mecanismo de palabra clave (contraseña), la identificación requiere:
  - La palabra clave proporcionada por el usuario se compare con la del sistema
- Los usuarios podrían identificarse en cualquier localidad → distintas implementaciones:
  - Las palabras clave se replican en todas las localidades

    Inconveniente: compromete la seguridad de las palabras clave porque es más fácil que un intruso acceda a ellas (atacando la menos protegida)

SEGURIDAD EN BBDD DISTRIBUIDAS

Identificación del usuario

- Los usuarios podrían tener una localidad principal en la que realizan su identificación
  - El usuario se identifica a través de una localidad específica

  Inconveniente: las localidades confían en la identificación hecha por otras localidades: incompatible con la autonomía local (localidades dependen de otras localidades)

  Solución:

  - Restringir que cada usuario solo pueda identificarse en su localidad principal
  - Se podrían usar palabras clave para conectar desde localidades remotas a sus localidades principales (ej. Certificado digital)

SEGURIDAD EN BBDD DISTRIBUIDAS

Cumplimiento de las **reglas de autorización**

- Regulan quién puede realizar qué acciones sobre los objetos de la base de datos
- Problemas adicionales:
  - La distribución de las reglas, que forman parte del catálogo
    - Deben de estar disponible en todas las localidades
  - Distribución de los mecanismos para forzar su cumplimiento
    - Debe de garantizarse que cada localidad aplica correctamente las mismas reglas
- Dos posibles soluciones alternativas:
  - (1) Replicación completa de las reglas de autorización:
    - Consistente con considerar un sistema único -> se maneja como si fuera un sistema centralizado
    - Consistente con catálogos completamente replicados -> copia de las reglas en todas las localidades
    - Requiere mecanismos de distribución online para actualizaciones -> cambio en las reglas supone actualización
    - Permite que la autorización se compruebe antes de ejecutar la consulta o en la validación de una transacción

SEGURIDAD EN BBDD DISTRIBUIDAS

Cumplimiento de las reglas de autorización

- (2) Asignación de las reglas a las localidades de los objetos a los que se refieren:
  - Consistente con la autonomía local -> cada localidad gestiona las reglas de forma independiente
  - Consistente con catálogos locales y no incurre en el problema de la actualización de la otra alternativa. Cada sitio mantiene solo las reglas necesarias para sus propios datos
  - La invalidación de los usuarios sin autorización a los objetos se realiza cuando se accede a los mismos
  - Las reglas se almacenan solo en las localidades donde están los datos, por lo que algunas localidades no siempre van a tener acceso directo a las reglas. La verificación se hace en la localidad donde están los datos, una vez que la consulta llega ahí.
  - Si un usuario no tiene permiso, se invalida la consulta en ese momento

SEGURIDAD EN BBDD DISTRIBUIDAS

Clases de usuarios

- Los usuarios se agrupan en clases con los mismos privilegios de acceso
- Se aplican las siguientes consideraciones:
  - Una clasificación natural de los usuarios:
    - Inducida por la distribución de la base de datos en las diferentes localidades

      Ejemplo: Es probable que "todos los usuarios de la localidad X" tengan los mismos privilegios

      Ejemplo: Los empleados de una sucursal bancaria comparten permisos para acceder a los datos
    - Se puede proporcionar un mecanismo explícito para nombrar esta clase
      - El sistema permite definir estas clases explícitamente: "usuarios_sucursal_madrid"

SEGURIDAD EN BBDD DISTRIBUIDAS

Clases de usuarios

- Problemas cuando los grupos incluyen usuarios de múltiples localidades:
  - ¿Dónde se almacena la información sobre los derechos de acceso de los grupos y qué reglas se deben usar para manipularlos?
  - Cualquier decisión concerniente al grupo no puede hacerse de manera autónoma →implica el consenso de la mayoría o totalidad de las localidades implicadas, o bien la decisión de un administrador de alto nivel